

The Gravity of Language

A Journey into Dependency Grammar and Parsing

Based on the lectures of Dr. Chetana Gavankar

Contents

1	Introduction: The Telescope Dilemma Returns	2
2	Part 1: The Solar System of Words	2
2.1	Head, Dependent, and Label	2
2.2	Formal Conditions of a Dependency Graph	3
3	Part 2: How Do We Build the Tree?	3
4	Part 3: Transition-Based Parsing (The Assembly Line)	3
4.1	The Configuration	4
4.2	The Four Actions (Arc-Eager Parsing)	4
4.3	Example Trace: "He sent her a letter ."	4
4.4	Training the Oracle (Machine Learning)	4
5	Part 4: Graph-Based Parsing (The Map Maker)	5
5.1	The Chu-Liu-Edmonds Algorithm	5
5.2	Learning in Graph Models (Global Update)	6
6	Conclusion	6

1 Introduction: The Telescope Dilemma Returns

The Problem with Boxes

In previous lectures, we viewed sentences as a series of nesting boxes called **Phrase Structure (Constituency) Grammars**. We grouped words into Noun Phrases (NP), Verb Phrases (VP), and so on.

But consider the classic ambiguous sentence: **"I saw a girl with a telescope."**

- Did *you* use a telescope to see her?
- Or did the *girl* possess the telescope?

Constituency trees get incredibly messy trying to represent this. They build deep, complex brackets of nested phrases. If we expand the sentence to *"I saw the man on the hill in Texas with the telescope at noon on Monday,"* a standard CFG parser outputs an astonishing **132 different valid trees!** Furthermore, languages with free word order (like Hindi or Russian) completely break the rigid "boxes" of Phrase Structure rules.

We need a different way to look at language—one that doesn't care about rigid boundaries, but instead cares about **relationships**.

2 Part 1: The Solar System of Words

Analogy Time: The Solar System

Instead of packing words into boxes, **Dependency Grammar** treats a sentence like a solar system. The main verb is the Sun. It is the gravitational center of the sentence. The nouns (subjects, objects) are planets orbiting the Sun. Adjectives and determiners are moons orbiting those planets.

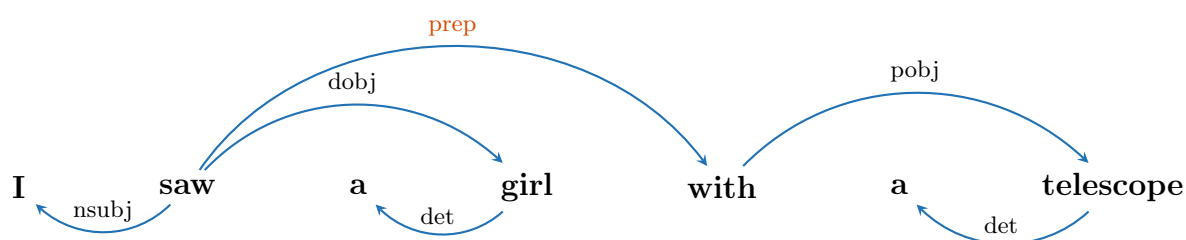
Every word (except the central Sun/ROOT) is gravitationally pulled by exactly *one* other word. We call these pulls **Dependencies**.

2.1 Head, Dependent, and Label

Dependency syntax postulates that syntactic structure consists of lexical items linked by binary, asymmetric relations ("arrows").

- **Head (Governor):** The core word. (The planet)
- **Dependent (Modifier):** The word that modifies or depends on the head. (The moon)
- **Label:** The type of grammatical relationship (e.g., subject, object).

Arrows are traditionally drawn from the **Head pointing to the Dependent**.



Notice how drawing the "prep" arrow from "saw" to "with" instantly clarifies that the seeing was done via the telescope. If the girl had the telescope, the arrow would point from "girl" to "with".

2.2 Formal Conditions of a Dependency Graph

A dependency structure is a directed graph $G = (V, A)$, where V are vertices (words) and A are arcs (dependencies). A valid dependency tree must be:

1. **Connected:** Every word is linked. There are no "floating" words.
2. **Acyclic:** If A depends on B, B cannot depend on A. No loops! ($i \rightarrow j \implies j \not\rightarrow^* i$).
3. **Single-Head Constraint:** Every word has *exactly one* incoming arrow. (Everyone has exactly one boss).
4. **Projective (Usually):** If you draw the words in a line and draw arrows above them, the arrows *should not cross*. (Note: Non-projective parses exist, especially in free-word-order languages, but projectivity makes algorithmic parsing much easier).

Key Concept: Universal Dependencies (UD)

Because every language is different, researchers created the **Universal Dependencies** project. It provides a uniform annotation scheme across many languages. It establishes universal Part-of-Speech tags and standard dependency relations (like *nsubj*, *dobj*, *amod*, *tmod*), allowing NLP models to learn universal grammatical truths regardless of the language being spoken.

3 Part 2: How Do We Build the Tree?

There are four main methods to perform Dependency Parsing:

1. **Dynamic Programming:** Like the CKY algorithm for phrase structures (e.g., Eisner's $O(n^3)$ algorithm).
2. **Constraint Satisfaction:** Eliminates edges that violate hard grammar rules.
3. **Deterministic Parsing:** A greedy, fast, left-to-right approach guided by Machine Learning (**Transition-Based Parsing**).
4. **Graph Algorithms:** Treats the sentence as a fully connected graph and finds the **Maximum Spanning Tree (Graph-Based Parsing)**.

We will focus deeply on the two most popular: Transition-Based and Graph-Based.

4 Part 3: Transition-Based Parsing (The Assembly Line)

Analogy Time: The Factory Conveyor Belt

Imagine an assembly line. Words enter on a conveyor belt called the **Buffer**. Next to it is a workbench called the **Stack**. A robotic arm looks at the word on the top of the Stack and the word at the front of the Buffer. It decides to either attach an arrow between them or shift words around. It makes this decision greedily, one step at a time, until the Buffer is empty.

4.1 The Configuration

A parser configuration is a snapshot of the factory floor, defined by $C = (S, B, A)$:

- **S**: The Stack of partially processed words. (Initially empty except for [ROOT])
- **B**: The Buffer of remaining input words. (Initially holds the whole sentence).
- **A**: The set of formed Arcs (dependencies). (Initially empty).

4.2 The Four Actions (Arc-Eager Parsing)

The machine learning "Oracle" looks at the Stack (S_{top}) and Buffer (B_{first}) and chooses one of 4 moves:

1. **Shift (SH)**: Move the front word of the Buffer onto the Stack.
2. **Left-Arc (LA)**: Create an arrow from the front of the Buffer to the top of the Stack. (The Buffer word is the head). Then, **pop** the Stack.
3. **Right-Arc (RA)**: Create an arrow from the top of the Stack to the front of the Buffer. (The Stack word is the head). Then, **push** the Buffer word onto the Stack.
4. **Reduce (RE)**: Pop the top word off the Stack (used when a word has found its head and all its dependents).

4.3 Example Trace: "He sent her a letter ."

Let's watch the assembly line build the sentence.

Transition	Stack (S)	Buffer (B)	New Arc Added
<i>Initial</i>	[]	[He, sent, her, a...]	-
Shift	[He]	[sent, her, a...]	-
Left-Arc	[]	[sent, her, a...]	sent $\xrightarrow{\text{SBJ}}$ He
Shift	[sent]	[her, a, letter...]	-
Right-Arc	[sent, her]	[a, letter, .]	sent $\xrightarrow{\text{IOBJ}}$ her
Shift	[sent, her, a]	[letter, .]	-
Left-Arc	[sent, her]	[letter, .]	letter $\xrightarrow{\text{DET}}$ a
Reduce	[sent]	[letter, .]	- (Done with 'her')
Right-Arc	[sent, letter]	[.]	sent $\xrightarrow{\text{DOBJ}}$ letter
Reduce	[sent]	[.]	- (Done with 'letter')
Right-Arc	[sent, .]	[]	sent $\xrightarrow{\text{PUNC}}$.

4.4 Training the Oracle (Machine Learning)

How does the parser know which of the 4 actions to take? We train a linear classifier using a **Treebank** (sentences annotated with gold-standard correct trees).

We extract **Features** $f(c, t)$ from the current configuration. For example:

- What is the POS of the top of the Stack? (e.g., Noun)
- What is the POS of the front of the Buffer? (e.g., Verb)

The Engineering Math: Perceptron Weight Update

The model computes a score for each action using a weight vector $\mathbf{w}$ and greedily picks the highest one: $t^* = \arg \max_t (\mathbf{w} \cdot f(c, t))$.

If it makes a mistake during training (predicts t^* instead of the true Oracle action t_o), it updates its weights online:

$$\mathbf{w}_{new} = \mathbf{w}_{old} + f(c, t_o) - f(c, t^*)$$

We literally add the features of the correct move to the weights, and subtract the features of the bad move!

Example Walkthrough (Weight Update): Let's say our features are [Empty Stack, Stack=Noun/Buffer=Verb, Stack=Verb/Buffer=Noun]. Weights $\mathbf{w} = [5.0, 5.0, 5.0]$. We have a special Left-Arc bias of +0.5, so $\mathbf{w} = [5.5, 5.5, 5.5]$ for LA. If the model wrongly predicts LA when the Oracle wanted Shift, we apply the update rule.

$$\mathbf{w}_{new} = \mathbf{w}_{old} + f(\text{Shift}) - f(\text{LA})$$

Over iterations, the weights naturally adjust to prefer the right transitions.

5 Part 4: Graph-Based Parsing (The Map Maker)

Transition-based parsing is incredibly fast ($O(N)$), but because it is greedy (never looks back), it can make early mistakes that cascade and ruin the rest of the sentence.

Graph-Based Parsing takes a global view. It treats parsing like drawing the most efficient road network across a country.

Key Concept: Maximum Spanning Tree (MST)

We create a fully connected directed graph where every word points to every other word. We use Machine Learning to assign a "score" (weight) to every possible arrow based on how grammatically likely it is.

The goal is to find the **Maximum Spanning Tree**: a subgraph that connects all nodes, has no cycles, gives every node exactly one head, and has the *highest total weight* possible.

$$G' = \arg \max_{G \in T(G_x)} \sum_{(i,j) \in G} w_{i,j}$$

5.1 The Chu-Liu-Edmonds Algorithm

Finding an MST in an undirected graph is easy (e.g., Kruskal's). Finding an MST in a *directed* graph is tricky because of the "single-head" constraint. We use the clever **Chu-Liu-Edmonds** algorithm.

Step 1: Greedy Selection.

For every word, simply pick the incoming arrow that has the highest score.

- If this creates a perfect tree, you are done! It is the MST.
- If this creates a cycle (a loop), proceed to Step 2.

Step 2: Cycle Contraction.

If a cycle is found (e.g., John $\rightarrow$ saw $\rightarrow$ John), the algorithm treats the entire cycle as a **single mega-node** (contraction).

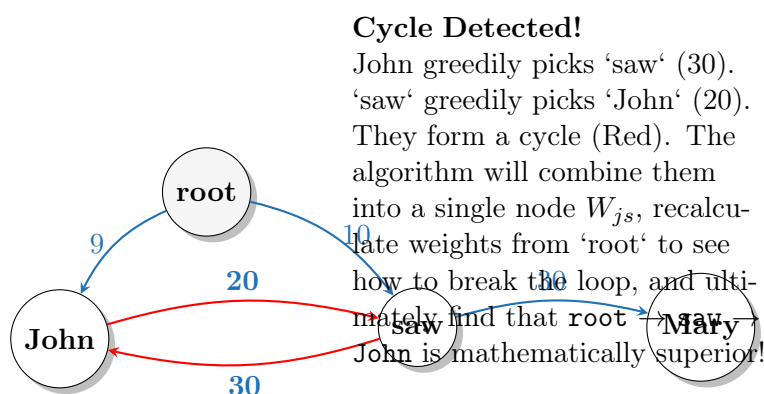
Step 3: Recalculate Weights.

We recalculate the weights of arrows entering and leaving this new mega-node.

- **Outgoing:** The weight is simply the max of the outgoing arcs from any node inside the cycle.
- **Incoming:** The new weight equals the value of the new external arrow *minus* the cost of breaking the internal cycle arrow it replaces.

Step 4: Recursive Unpacking.

We repeat the algorithm on the contracted graph until a valid tree is formed. Then, we "unpack" the mega-nodes, breaking the internal edge inside the cycle to allow the new optimal external edge in!

Chu-Liu-Edmonds Visualized**5.2 Learning in Graph Models (Global Update)**

Just like Transition-based models, Graph-based models use Machine Learning to figure out the weights of the arcs using features (like POS tags, direction, and distance). However, Graph models update weights **globally**:

$$\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + f(G_{true}) - f(G')$$

It compares the entire predicted MST (G') against the Gold Standard tree (G_{true}). If they differ, it adds the features of the correct tree to the weights, and subtracts the features of its bad prediction, learning to balance the entire sentence holistically.

6 Conclusion

Dependency parsing shifts our perspective from nested phrases to dynamic, word-to-word relationships.

- **Dependencies** map the grammatical pull of words, creating connected, acyclic, single-headed trees that elegantly handle ambiguity and free word order.
- **Transition-Based Parsers** (like Arc-Eager MaltParser) operate like a fast, greedy assembly line using a Stack and a Buffer.

- **Graph-Based Parsers** (like MSTParser) take a global view, using the Chu-Liu-Edmonds algorithm to mathematically guarantee the optimal Maximum Spanning Tree.
- Both systems rely on rich **Feature Vectors** and **Machine Learning classifiers** (using structured perceptron updates) to navigate the immense complexity of human language.